

Implementation Note: Direct Gaussian QEq and External-Field Regularization

SchnetPack charge-prediction implementation

July 29, 2026

Abstract

This note relates the implementation of `EMLEQEqStatic` in `src/schnetpack/atomistic/atomwise.py` to the underlying constrained charge-equilibration (QEq) equations. The implementation constructs a dense Gaussian Coulomb matrix and solves the charge constraint directly with `torch.linalg.solve`. It computes both a vacuum charge distribution and, when an external electrostatic potential is supplied, a polarized charge distribution. The appendix documents the auxiliary field-regularized training task implemented in `src/schnetpack/task.py` and configured in `train_field_regularized.yaml`.

1 Notation and units

For one structure containing N atoms, let

- q_i be the total charge on atom i , in units of the elementary charge e ;
- χ_i be its environment-dependent electronegativity-like coefficient, in $\text{kJ mol}^{-1} e^{-1}$;
- ϕ_i be the external electrostatic potential at atom i , in $\text{kJ mol}^{-1} e^{-1}$;
- σ_i be the Gaussian charge width, in $\text{\AA}$;
- Q be the prescribed total charge of the structure, in e ; and
- A be the QEq interaction matrix, in $\text{kJ mol}^{-1} e^{-2}$.

Vectors are denoted by bold symbols, and $\mathbf{1}$ is the vector containing N ones. The implementation uses a Coulomb conversion constant $k_e = \text{K_E_KJ MOL_ANG_E2}$ so that distances in $\text{\AA}$ produce energies in the units above.

2 The quadratic QEq problem

In vacuum, the charge-dependent energy is

$$E_0(\mathbf{q}) = \boldsymbol{\chi}^\top \mathbf{q} + \frac{1}{2} \mathbf{q}^\top A \mathbf{q}, \quad (1)$$

subject to the total-charge constraint

$$\mathbf{1}^\top \mathbf{q} = Q. \quad (2)$$

The linear term favors transfer according to the learned electronegativities, whereas the quadratic term penalizes self-charging and describes interatomic Coulomb interactions.

If an external potential is present, its coupling to the charges is

$$E_{\text{ext}}(\mathbf{q}) = \boldsymbol{\phi}^\top \mathbf{q}. \quad (3)$$

The polarized QEq energy therefore becomes

$$E_\phi(\mathbf{q}) = (\boldsymbol{\chi} + \boldsymbol{\phi})^\top \mathbf{q} + \frac{1}{2} \mathbf{q}^\top \mathbf{A} \mathbf{q}, \quad (4)$$

with the same constraint in Eq. (2). The sign follows the implementation convention: the external potential is added to $\boldsymbol{\chi}$ before solving.

3 Learned parameters and Gaussian widths

The neural-network parameter head produces an atom-wise valence width s_i and an electronegativity χ_i . A learnable global scale a_{QEq} converts the former into the QEq width,

$$\sigma_i = a_{\text{QEq}} s_i. \quad (5)$$

The implementation is, schematically,

```
1 s = inputs[self.widths_key]
2 chi = inputs[self.chi_key]
3 a_qeq = self._get_a_qeq()
4 sigma = a_qeq * s
```

Optional lower and upper bounds are then applied to σ_i . These bounds are numerical and physical robustness controls: a very large width yields a small self-hardness and can make the model excessively polarizable, whereas a vanishing width produces a very large hardness.

4 Construction of the QEq matrix

4.1 Diagonal self-hardness

For the Gaussian convention used here, the diagonal term is

$$A_{ii} = J_{ii} = \frac{k_e}{\sqrt{\pi} \sigma_i}. \quad (6)$$

The corresponding code is

```
1 Jii = K_E_KJMOL_ANG_E2 / (sigma * math.sqrt(math.pi))
2 if jii_floor > 0.0:
3     Jii = Jii + jii_floor
```

An optional positive `jii_floor` increases every diagonal entry and damps the response to an external field.

4.2 Off-diagonal Gaussian Coulomb interaction

For atoms $i \neq j$, define

$$R_{ij} = \|\mathbf{R}_i - \mathbf{R}_j\|, \quad (7)$$

$$\sigma_{ij} = \sqrt{\sigma_i^2 + \sigma_j^2}. \quad (8)$$

The off-diagonal matrix element is

$$A_{ij} = J_{ij} = k_e \frac{\text{erf}(R_{ij}/(\sqrt{2}\sigma_{ij}))}{R_{ij}}. \quad (9)$$

This is implemented by

```

1 dR = R[:, None, :] - R[None, :, :]
2 Rij = torch.linalg.norm(dR, dim=-1)
3 sigma_ij = torch.sqrt(
4     sigma[:, None] ** 2 + sigma[None, :] ** 2
5 ).clamp_min(self.eps_sigma)
6
7 Jij = (
8     K_E_KJMOL_ANG_E2
9     * torch.erf(Rij_safe / (math.sqrt(2.0) * sigma_ij))
10    / Rij_safe
11 )
12 Jij = Jij.masked_fill(eye, 0.0)
13 A = torch.diag(Jii) + Jij

```

Gaussian smearing keeps the interaction finite at short range. At large separation, $\text{erf}(x) \rightarrow 1$, so Eq. (9) approaches the point-charge interaction k_e/R_{ij} . If `offdiag_cutoff` is set, off-diagonal terms beyond that distance are set to zero. The present training configuration uses a cutoff of 10 Å.

5 Constrained direct solution

Introduce a Lagrange multiplier λ and the Lagrangian

$$\mathcal{L}(\mathbf{q}, \lambda) = \mathbf{g}^\top \mathbf{q} + \frac{1}{2} \mathbf{q}^\top \mathbf{A} \mathbf{q} + \lambda (\mathbf{1}^\top \mathbf{q} - Q), \quad (10)$$

where $\mathbf{g} = \boldsymbol{\chi}$ in vacuum and $\mathbf{g} = \boldsymbol{\chi} + \boldsymbol{\phi}$ in the external field. Stationarity gives

$$\mathbf{A} \mathbf{q} + \mathbf{g} + \lambda \mathbf{1} = 0, \quad (11)$$

$$\mathbf{1}^\top \mathbf{q} = Q. \quad (12)$$

These equations form the augmented system

$$\underbrace{\begin{bmatrix} \mathbf{A} & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{bmatrix}}_K \begin{bmatrix} \mathbf{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} -\mathbf{g} \\ Q \end{bmatrix}. \quad (13)$$

The Python implementation constructs Eq. (13) literally:

```

1 n = A.shape[0]
2 K = torch.zeros((n + 1, n + 1), device=A.device, dtype=A.dtype)
3 K[:n, :n] = A
4 K[:n, n] = 1.0
5 K[n, :n] = 1.0
6
7 b = torch.zeros((n + 1,), device=A.device, dtype=A.dtype)
8 b[:n] = -g
9 b[n] = Q
10 return torch.linalg.solve(K, b)[:n]

```

The final element of the full solution is λ ; slicing with `[:n]` returns only the atomic charges.

This is a dense direct linear solve, not an iterative self-consistency loop. Consequently, there is no QEq convergence tolerance or maximum iteration count. The mathematical stationary point is obtained up to floating-point roundoff and amplification by the condition number of K . The augmented matrix is indefinite because of the constraint row and column, but it is solvable when the constrained QEq problem is well posed.

6 Vacuum and polarized passes

Each structure in a batch is solved separately. The vacuum and polarized calls are

```

1 A = self._build_qeq_matrix(Rm, sigmam, Jiim)
2 q0m = self._solve_qeq(A, chim, Qm)
3 qphim = (
4     self._solve_qeq(A, chim + phim, Qm)
5     if polarize
6     else q0m
7 )

```

Thus, $\mathbf{q}_0$ solves

$$A\mathbf{q}_0 + \boldsymbol{\chi} + \lambda_0\mathbf{1} = 0, \quad \mathbf{1}^\top \mathbf{q}_0 = Q, \quad (14)$$

whereas $\mathbf{q}_\phi$ solves

$$A\mathbf{q}_\phi + \boldsymbol{\chi} + \boldsymbol{\phi} + \lambda_\phi\mathbf{1} = 0, \quad \mathbf{1}^\top \mathbf{q}_\phi = Q. \quad (15)$$

The matrix A is identical in the two solves; only the linear term changes. Adding the same constant to all components of $\boldsymbol{\phi}$ does not change the charge distribution under an exact constrained solve. It changes only the Lagrange multiplier. This gauge invariance motivates molecule-wise centering of fields in the training augmentation described in Appendix A.

7 Finite-precision charge correction and warning

Although Eq. (13) contains the total-charge constraint, finite-precision arithmetic can leave a small residual. For each structure the implementation computes

$$\Delta q = \frac{Q - \sum_i q_i}{N} \quad (16)$$

and shifts every charge uniformly,

$$q_i \leftarrow q_i + \Delta q. \quad (17)$$

The code is

```

1 q_sum = snn.scatter_add(q, idx_m, dim_size=maxm)
2 dq = (Q - q_sum) / natoms.unsqueeze(-1)
3 q = q + dq[idx_m]

```

Even after this operation, the subsequent floating-point summation can retain a small residual. The code warns if either

$$r_0 = \max_m \left| \sum_{i \in m} q_{0,i} - Q_m \right| \quad \text{or} \quad r_\phi = \max_m \left| \sum_{i \in m} q_{\phi,i} - Q_m \right| \quad (18)$$

exceeds $10^{-5}e$. This value is a post-solve diagnostic threshold, not a solver convergence criterion. A large or repeated residual can indicate an ill-conditioned QEq matrix, extreme charge cancellation, or insufficient floating-point precision. The configured training precision is 32 bit.

8 External-field embedding energy

The implementation reports

$$E_{\text{emb}} = \frac{1}{2}(\mathbf{q}_0 + \mathbf{q}_\phi)^\top \boldsymbol{\phi}, \quad (19)$$

using

```

1 embedding_atom = 0.5 * (q0 + qphi) * phi
2 embedding_energy = scatter_add(embedding_atom, idx_m)

```

Equation (19) is the linear-response charging expression: as the external potential is turned on, the charge changes linearly from q_0 to q_ϕ , whose path average is $(q_0 + q_\phi)/2$.

9 Core and valence diagnostic outputs

Only after solving QEq does the implementation look up an element-dependent core charge,

$$q_{\text{val},i}^\phi = q_{\phi,i} - q_{\text{core},i}, \quad (20)$$

$$q_{\text{val},i}^0 = q_{0,i} - q_{\text{core},i}. \quad (21)$$

The corresponding code is

```

1 q_core = self._get_qcore(Z).unsqueeze(-1)
2 q_val = qphi - q_core
3 q_val_vac = q0 - q_core

```

Therefore, `qcore_table` does not enter the QEq matrix, linear term, total-charge constraint, charge solution, or embedding energy. It affects only the reported core/valence decomposition. In the present configuration the table contains 1.0 for H, 6.35 for O, and 4.46 for Fe.

10 Computational characteristics

For N QEq atoms, explicitly storing the dense matrix costs $\mathcal{O}(N^2)$ memory and its direct factorization costs approximately $\mathcal{O}(N^3)$ work. This approach is attractive for small and medium-sized QEq regions because it has no iterative convergence error and is directly differentiable through PyTorch. For very large systems, a sparse or matrix-free iterative method can be faster and use less memory. A direct solve is not automatically immune to numerical error: poor conditioning can amplify roundoff in either direct or iterative methods.

A Field-regularized training task

A.1 Purpose and two-pass training structure

`FieldRegularizedAtomisticTask` augments ordinary supervised training with a second, unlabeled external-field pass. The purpose is to discourage unphysical charge magnitudes and excessive field-induced charge transfer when field-dependent reference labels are unavailable.

For every training batch, the task performs:

1. a zero-field forward pass, used for the ordinary supervised loss; and
2. an augmented-field forward pass, used only for charge regularization.

When field regularization is disabled, the class delegates directly to the standard `AtomisticTask`. The override is confined to `training_step`; validation and testing follow the normal task path and do not receive random field augmentation.

A.2 Supervised zero-field pass

The task makes a shallow copy of the batch and explicitly sets the external potential to zero at every atom:

```

1 zero_batch = dict(batch)
2 zero_batch[self.phi_key] = batch[properties.Z].new_zeros(
3     batch[properties.Z].shape, dtype=batch[properties.R].dtype
4 )
5 pred_zero = self.predict_without_postprocessing(zero_batch)
6 q_zero = pred_zero[self.charges_key]
7 supervised_loss = self.loss_fn(pred_zero, targets)

```

For the supplied training configuration, the supervised composite loss is

$$\mathcal{L}_{\text{sup}} = 0.05 \text{MSE}(V) + 0.70 \text{MSE}(\mathbf{F}) + 0.15 \text{MSE}(\mathbf{q}_0) + 0.10 \text{MSE}(\mathbf{s}). \quad (22)$$

Here V is V_{BuRNN} , $\mathbf{F}$ is $\mathbf{F}_{\text{BuRNN}}$, and $\mathbf{s}$ denotes `mbis_valence_widths`. The implementation stores `q_zero` before output constraints are applied, because such constraints may replace entries in the prediction dictionary.

A.3 Generation of the augmented field

Two field modes are supported:

random Draw an independent value for each atom, $\tilde{\phi}_i \sim \mathcal{N}(0, \sigma_\phi^2)$.

batch Read the field from `phi_static_aug`, falling back to `phi_static`; if neither is present, generate a random field.

In random mode the implementation starts from

```

1 source = torch.randn_like(batch[properties.R][:, 0]) * self.phi_std

```

If molecule-wise centering is enabled, it applies

$$\phi_i \leftarrow \tilde{\phi}_i - \frac{1}{N_m} \sum_{j \in m} \tilde{\phi}_j, \quad i \in m. \quad (23)$$

This removes the physically redundant constant-potential component separately for every molecule. The result is then clipped component-wise:

$$\phi_i \leftarrow \text{clip}(\phi_i, -\phi_{\text{max}}, \phi_{\text{max}}). \quad (24)$$

Clipping after centering can reintroduce a small nonzero molecular mean when individual components hit the bounds; this has no effect on exact constrained QEq charge differences, although finite-precision effects remain possible.

The current configuration uses

$$\sigma_\phi = 300 \text{ kJ mol}^{-1} \text{ e}^{-1}, \quad \phi_{\text{max}} = 1000 \text{ kJ mol}^{-1} \text{ e}^{-1}, \quad (25)$$

with centering enabled.

A.4 Auxiliary polarized pass

The field is placed under the same key read by `EMLEQEqStatic`, and the model is evaluated again:

```

1 aug_batch = dict(batch)
2 aug_batch[properties.R] = batch[properties.R].detach().clone()
3 aug_batch[self.phi_key] = phi_aug
4 pred_aug = self.predict_without_postprocessing(aug_batch)
5 q_phi = pred_aug[self.charges_key]

```

The detached copy of the positions keeps force derivatives in this auxiliary forward pass independent of the supervised force graph. No auxiliary energy or force label is used. Gradients from the charge regularizers still propagate through $\mathbf{q}_\phi$ into the QEq parameter head and the shared neural-network representation.

A.5 Element-dependent charge-bound penalty

Let $b_i = b(Z_i)$ be the configured absolute charge bound for atom i . The penalty is a squared hinge loss,

$$\mathcal{L}_{\text{bound}} = \frac{1}{N_{\text{batch atoms}}} \sum_i [\max(|q_{\phi,i}| - b_i, 0)]^2. \quad (26)$$

It is zero inside the allowed interval and grows quadratically outside it. The configured bounds are

Element	Atomic number	$b(Z)$ in e
H	1	0.8
O	8	1.5
Fe	26	3.2
Other elements	—	2.0

The bound is on the total QEq charge $q_{\phi,i}$, not on the diagnostic valence charge derived from `qcore_table`.

A.6 Field-induced charge-change penalty

The second auxiliary term is

$$\mathcal{L}_{\Delta q} = \frac{1}{N_{\text{batch atoms}}} \sum_i (q_{\phi,i} - \text{stopgrad}(q_{0,i}))^2. \quad (27)$$

The code uses

```
1 dq_loss = (q_phi - q_zero.detach()).square().mean()
```

Detaching $\mathbf{q}_0$ means this auxiliary term treats the zero-field charges as a fixed reference during backpropagation. It pushes only the polarized branch toward that reference and does not use the regularizer to move the reference charges themselves. The term limits field sensitivity, but it is not an explicit Jacobian norm such as $\|\partial \mathbf{q} / \partial \phi\|^2$. The configuration key `w_sensitivity` is reserved for such a future penalty and must currently be zero.

A.7 Complete training objective

The final per-batch objective is

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + w_{\text{bound}} \mathcal{L}_{\text{bound}} + w_{\Delta q} \mathcal{L}_{\Delta q} \quad (28)$$

with the present weights

$$w_{\text{bound}} = 0.01, \quad w_{\Delta q} = 0.001. \quad (29)$$

Both regularizers are unsupervised: they require neither polarized reference charges nor field-dependent energies and forces. Their numerical influence depends on their raw scales as well as the displayed weights, so logged loss components should be inspected rather than interpreting the weights alone.

A.8 Logged diagnostics

During training, the task logs the following batch-level diagnostics:

- maximum polarized absolute charge, `train_q_phi_abs_max`;
- unweighted bound penalty, `train_q_phi_bound_loss`;

- unweighted charge-change penalty, `train_q_phi_dq_loss`; and
- minimum, maximum, and population standard deviation of the augmented potential.

These quantities make it possible to distinguish an overly strong field distribution from an overly responsive QEq model and to assess the actual scale of each auxiliary contribution to Eq. (28).

A.9 Current configuration summary

Setting	Current value
Task	<code>FieldRegularizedAtomisticTask</code>
Field key read by QEq	<code>phi_static</code>
Augmentation mode	<code>random</code>
Random-field standard deviation	300 kJ mol ⁻¹ e ⁻¹
Absolute clip	1000 kJ mol ⁻¹ e ⁻¹
Molecule-wise centering	enabled
w_{bound}	0.01
$w_{\Delta q}$	0.001
Explicit Jacobian penalty	not implemented; weight fixed to zero
Training precision	32 bit